ELEC 475 Lab 3 Post Lab

ELEC 475, Fall 2026

Oct 22nd, 2026

Josh Westlake (20349406)

Matthew Gilbreath (20353507)

GitHub repository: https://github.com/MatthewGilbreath/elec475-lab2

1. Model

This lab builds up pet nose localization using a variety of different architectures. The main lab focused on developing the custom SnoutNet, SnoutNetAlexNet (SnoutNet-A), and SnoutNetVGG16 (SnoutNet-V) regressors plus prediction ensembling, which were each evaluated by Euclidean pixel error and a CED curve. During the post-lab, we continued to build on these networks by implementing a second-stage correction network that refines any of those first-stage predictions from a cropped patch.

1.1 Correction Stage: Training Patches

CorrectionDataset.py is designed to build a random 64x64 training patch per training image, per epoch. The patch center is sampled as the true nose plus uniform noise in [-40,40] px in u and v, to simulate a noisy first-stage prediction. The patch is then cropped into a 128x128 region around the center from the original image (setting all out-of-bounds pixels to black) and then resized to a 64x64 image with bilinear interpolation. The regression target is the offset from the patch center to the true nose (du, dv), which is then normalized. When augment=True, a horizontal flip (p=0.5) mirrors the patch and negates du, and a color jitter is applied; and targets get updated if needed.

1.2 Correction Stage: Architecture

The correction architecture takes a 3,64,64 ImageNet-normalized patch and outputs a 2-value normalized offset. The base correctionNet architecture is described below in Table 1.

Table 1 Correction Stage Model Architecture

Stage	Output Shape
Input	(3,64,64)
Block 1 (32 ch, conv 3x3 + BN + ReLU + 2x2 pool)	(32,32,32)
Block 2 (64 ch)	(64,16,16)
Block 3 (128 ch)	(128,8,8)
Block 4 (128 ch)	(128,4,4)
Flatten	128*4*4 = 2048
FC1 + ReLU	2048 -> 256
FC2	256 -> 2

1.3 CorrectionNet Modified Architecture

CorrectionNetModified keeps the same four blocks, channel counts, pooling and fully connected layers as CorrectionNet, but each block contains two convolution layers instead of one. Each block applies a 3×3 convolution, batch normalization and ReLU twice before the 2×2 max pooling. This doubles the number of convolution layers from four to eight and increases the parameter count from 766,594 to 1,108,642. Following the VGG approach, two stacked 3×3 convolutions

cover the same area as one 5×5 convolution with an extra nonlinearity in between, giving each block more depth to build features before the patch is downsampled.

Table 2 Correction Stage Model Modified Architecture

Stage	Output shape
Input	(3, 64, 64)
Block 1 (32 ch, [conv 3×3 + BN + ReLU] × 2, then 2×2 max pool)	(32, 32, 32)
Block 2 (64 ch, same pattern)	(64, 16, 16)
Block 3 (128 ch, same pattern)	(128, 8, 8)
Block 4 (128 ch, same pattern)	(128, 4, 4)
Flatten	128 × 4 × 4 = 2048
FC1 + ReLU	2048 → 256
FC2	256 → 2

2. Training

All four correction models were trained with train_correction.py using the same instructions. The runs differ by the model class (CorrectionNet or CorrectionNetModified, selected with --modified) and whether augmentation is enabled (disabled with --no-augment). Table 3 shows a summary of the four training runs.

Table 3 The four correction-model training runs.

Run	Model	Command flags	Training time (CPU)
correction_noaug	CorrectionNet	--no-augment	319.9 s
correction_aug	CorrectionNet	(augmentation on)	337.3 s
correction_mod_noaug	CorrectionNetModified	--modified --no-augment	1165.3 s
correction_mod_aug	CorrectionNetModified	--modified (augmentation on)	1078.5 s

2.1 Training data and patch sampling

The correction models were trained only on the 5,494 images in the training set. The test images were never used during training, and no separate validation set was used.

Because a training image has no first-stage prediction to correct, one was simulated. Each time an image was loaded, the patch centre was placed at the true nose position plus a random offset of up to 40 pixels in each direction. A new random offset was drawn per image every epoch.

A 128 x 128 pixel region around the patch centre was cropped from the original image, with any area outside the image filled in black. The crop was resized to 64 × 64 and normalized with the same ImageNet statistics used for the first-stage models. The training target was the horizontal

and vertical distance from the patch centre to the true nose, divided by 64 to keep the values roughly between −1 and 1.

2.2 Augmentation

For the two augmented runs, two augmentations were applied to each patch:

- Horizontal flip, applied to half of the patches. Flipping moves the nose to the other side of the centre, so the horizontal part of the target was reversed to match. The vertical part was unchanged.
- Colour jitter, which randomly varied the brightness, contrast and saturation of each patch by up to ±30%.

The non-augmented runs used the same patch sampling without these two steps. No augmentation was used when the models were applied to the test set.

2.3 Optimization

During training, the following settings and hyperparameters were applied for each:

Table 4 Hyperparameters used during training.

Setting	Value
Loss	Mean squared error
Optimizer	Adam (default settings)
Initial learning rate	1e-3
Learning-rate schedule	Cosine annealing over 20 epochs
Weight decay	1e-5
Batch size	64 (shuffled every epoch)
Epochs	20
Random seed	42

Each model was trained for 20 epochs.

3. Outputs

Table 5 and Table 6 show the accuracy and cost of every model, as printed by evaluate.py.

Table 5 Localization error (px) on the 698 test images.

Model	Min	Mean	Median	Max	Std
In-Lab (first-stage models)					
Baseline (mean)	0.6	79.7	70.0	848.2	56.1

Model	Min	Mean	Median	Max	Std
SnoutNet	0.8	57.0	44.3	1084.5	57.4
SnoutNet-A	1.8	44.8	37.0	690.0	39.6
SnoutNet-V	0.4	12.8	8.7	175.5	17.3
Ensemble (mean)	0.7	31.2	23.5	597.2	33.7
Ensemble (median)	0.4	29.4	20.3	676.3	38.0
Corrected SnoutNet					
SnoutNet + corr.	0.2	43.8	19.4	1062.4	61.1
SnoutNet + corr. (aug)	0.2	43.5	20.1	1061.9	61.2
SnoutNet + mod. corr.	0.9	45.2	24.2	1081.4	61.2
SnoutNet + mod. corr. (aug)	0.1	40.8	14.3	1072.2	62.5
Corrected SnoutNet-V					
SnoutNet-V + corr.	0.1	12.2	7.7	198.3	17.0
SnoutNet-V + corr. (aug)	0.1	11.8	7.5	205.6	17.1
SnoutNet-V + mod. corr.	0.3	11.7	7.2	196.0	17.3
SnoutNet-V + mod. corr. (aug)	0.1	10.4	6.0	198.8	17.0
Corrected Ensemble (mean)					
Ensemble (mean) + corr.	0.4	20.4	10.3	595.5	34.4
Ensemble (mean) + corr. (aug)	0.6	20.5	9.6	594.5	34.6
Ensemble (mean) + mod. corr.	0.4	22.0	11.7	598.7	34.6
Ensemble (mean) + mod. corr. (aug)	0.2	17.8	7.3	594.1	34.8

Table 6 Parameter count and inference time per image

Model	Params	ms/img
In-Lab (first-stage models)		
Baseline (mean)	0	0.0
SnoutNet	5,617,794	12.6
SnoutNet-A	57,012,034	13.9
SnoutNet-V	134,268,738	97.3
Ensemble (mean)	196,898,566	123.8
Ensemble (median)	196,898,566	123.8
Corrected SnoutNet		
SnoutNet + corr.	6,384,388	13.8
SnoutNet + corr. (aug)	6,384,388	13.9
SnoutNet + mod. corr.	6,726,436	15.4
SnoutNet + mod. corr. (aug)	6,726,436	15.8
Corrected SnoutNet-V		
SnoutNet-V + corr.	135,035,332	98.6
SnoutNet-V + corr. (aug)	135,035,332	98.7

Model	Params	ms/img
SnoutNet-V + mod. corr.	135,377,380	100.2
SnoutNet-V + mod. corr. (aug)	135,377,380	100.2
Corrected Ensemble (mean)		
Ensemble (mean) + corr.	197,665,160	124.9
Ensemble (mean) + corr. (aug)	197,665,160	125.2
Ensemble (mean) + mod. corr.	198,007,208	127.3
Ensemble (mean) + mod. corr. (aug)	198,007,208	126.6

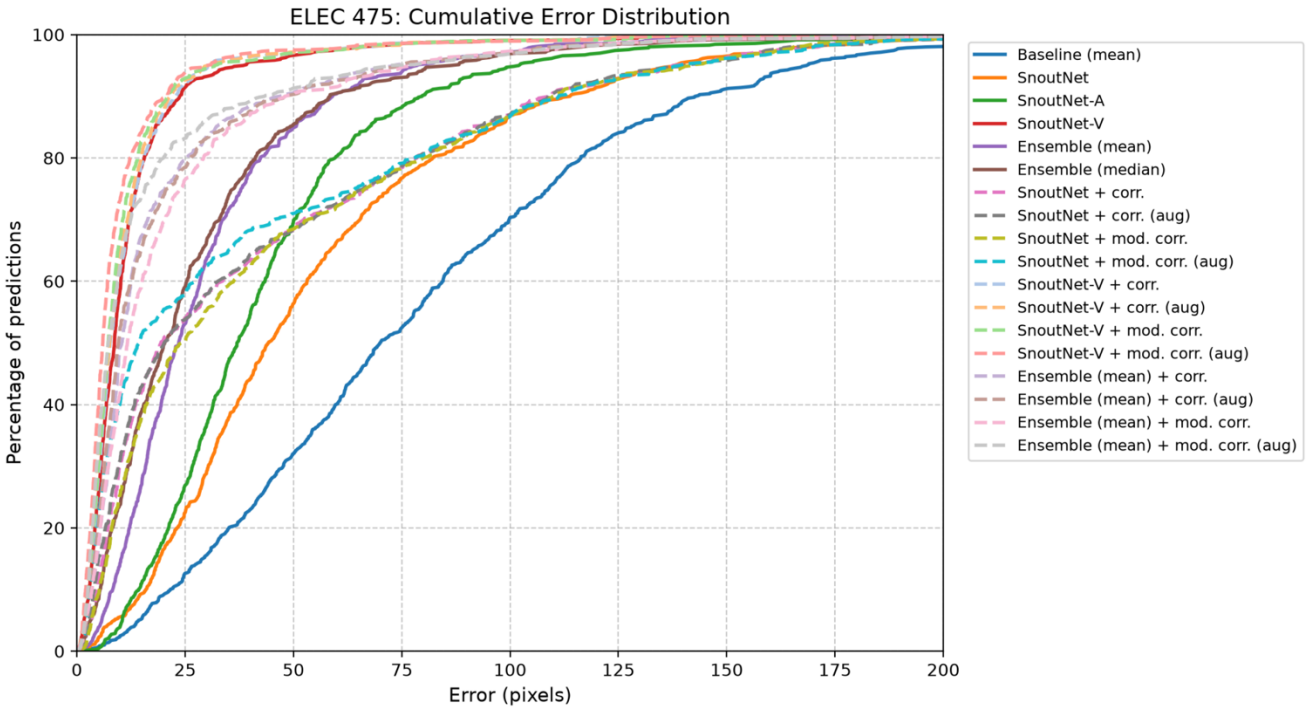


Figure 1 Cumulative Error Distribution: In-Lab results and the corrected SnoutNet, SnoutNet-V and mean ensemble, for both correction models, with and without augmentation

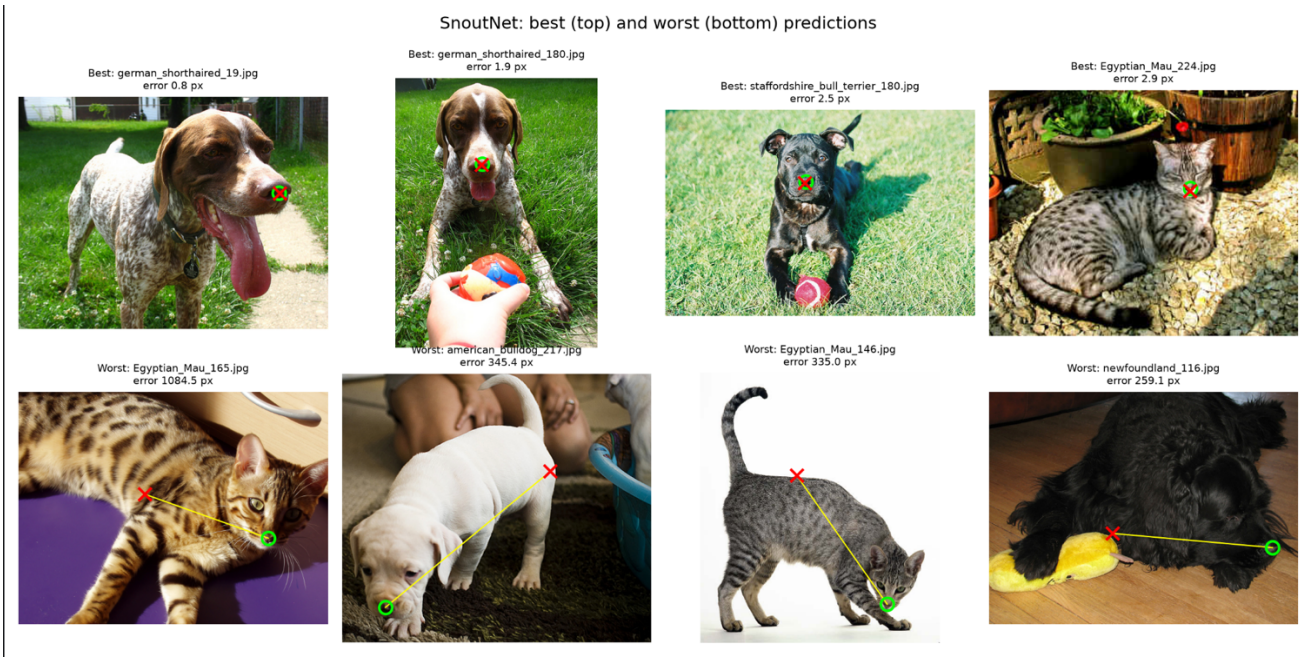


Figure 2 best_worst_snoutnet.png



Figure 3 best_worst_snoutnet_correction_noaug.png



Figure 4 best_worst_snoutnet_correction_aug.png

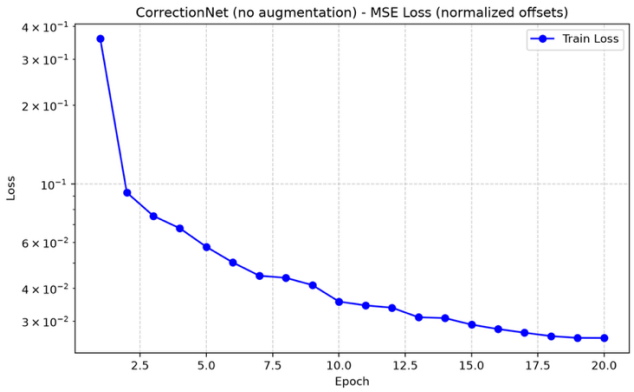


Figure 5 loss_correction_noaug.png

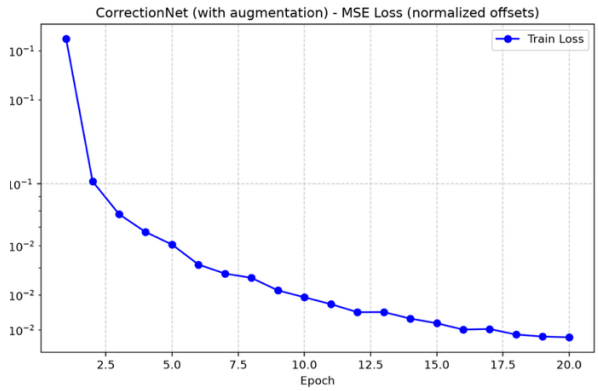


Figure 6 loss_correction_aug.png

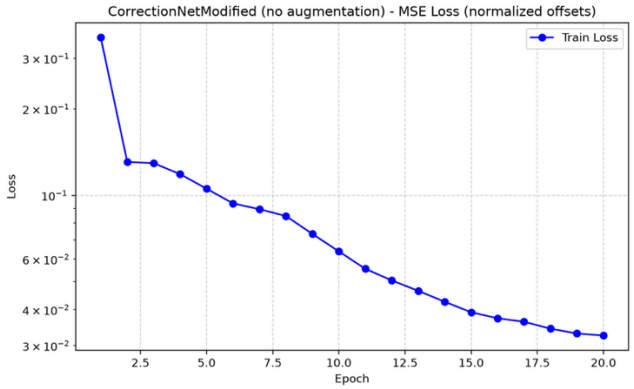


Figure 7 loss_correction_mod_noaug.png

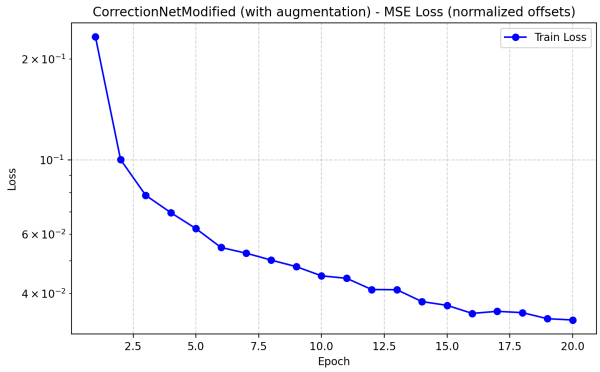


Figure 8 loss_correction_mod_aug.png

All four runs train smoothly. The two CorrectionNet runs level off from about epoch 15, while the two CorrectionNetModified runs are still improving at epoch 20. Without augmentation, the modified model stalls for the first few epochs and ends with a higher final loss than CorrectionNet (0.0325 vs 0.0259). With augmentation, it ends with the lowest loss of all four runs (0.0157, compared with 0.0282 for CorrectionNet with augmentation).

4. Analysis

4.1 Failure cases

The worst predictions (Figure 2, Figure 3, and Figure 4) tended to share a few features, each of which makes the nose harder to pick out:

- The nose is the same colour as the rest of the animal. With so little contrast, the network has few visual cues to separate the nose from the rest of the animal.
- Spotted or patterned fur. Dark spots look like a nose. The network sees several nose-like blobs and can select the wrong one.

- An obscured nose. When the nose is hidden by the animal's own body, turned away from the camera, or blocked by an object in the foreground, there is nothing to localize.

4.2 Pretraining

Pretraining gives the largest improvement. SnoutNet, trained from scratch, reaches a mean error of 57.0 px, only moderately better than the 79.7 px baseline. The pretrained SnoutNet-A and SnoutNet-V reach 44.8 px and 12.8 px.

ImageNet contains over a million labelled images, including more than a hundred dog breeds. Networks trained on it have already learned edges and fur textures in their early layers. A pretrained backbone therefore already recognizes pet faces and only has to learn how to turn those features into a nose position. SnoutNet has to learn everything from only 5,494 training images.

4.3 Ensembles

Averaging the three models yields worse results than SnoutNet-V alone. The ensemble does beat SnoutNet and SnoutNet-A, but those two models are far less accurate than SnoutNet-V. The median ensemble does slightly better because it ignores one outlier per coordinate, but when the two weaker models agree on a wrong position, they outvote SnoutNet-V.

An ensemble is most effective when its members are similarly accurate and make different mistakes, so their errors partly cancel. It is also a reasonable way to hedge when several models perform similarly on a validation set and there is no clear best choice. Here, SnoutNet-V is clearly better, so using it alone is the better option.

4.4 Correction model

The correction model does refine predictions, but only when the first prediction is already close to the nose. It was trained on patch centres up to 40 pixels from the nose, and each patch only covers a 128 x 128 region of the original image. When the first prediction is further off than that, the nose is not inside the patch and there is nothing to correct towards. This is why the worst predictions barely change.

The improvement is largest for SnoutNet, whose median error is more than halved, since most of its errors are moderate and fall within the patch. SnoutNet-V improves only slightly, because its predictions are already about as precise as the correction model itself.

4.5 Augmentation

Augmentation's effect depends heavily on which correction architecture it's applied to, and is much larger for the modified model. For base CorrectionNet, aug gives a small, mixed effect:

SnoutNet-V improves on both mean and median (12.2 → 11.8, 7.7 → 7.5), Ensemble improves on median but not mean (10.3 → 9.6 vs 20.4 → 20.5), and SnoutNet is essentially a wash (mean 43.8 → 43.5 slightly better, median 19.4 → 20.1 slightly worse).

For CorrectionNetModified, augmentation produces large, consistent improvements across all three base models and both metrics: SnoutNet's mean drops 45.2 → 40.8 and median 24.2 → 14.3; SnoutNet-V's mean 11.7 → 10.4, median 7.2 → 6.0; Ensemble's mean 22.0 → 17.8, median 11.7 → 7.3. Most of this difference comes from the modified model's no-augmentation run, which trained poorly from the start.

4.6 Modified correction model

During the modification process, a second 3×3 convolution is added as well as batch normalization and ReLU to each of the four blocks, giving eight convolution layers instead of four. Everything else is unchanged. The aim was to give each block more depth to build features before downsampling, following the VGG approach of stacking small convolutions.

Whether it improves performance depends on whether augmentation is used. Without augmentation, the modified architecture underperforms base CorrectionNet for two of three base models, SnoutNet's mean rises 43.8 → 45.2, Ensemble's mean rises 20.4 → 22.0, with only SnoutNet-V improving (mean 12.2 → 11.7).

With augmentation, the picture reverses completely: mod. corr. (aug) beats corr. (aug) for all three models and both metrics, often substantially (SnoutNet mean 43.5 → 40.8; SnoutNet-V mean 11.8 → 10.4; Ensemble mean 20.5 → 17.8). A likely reason is how the models trained. Without augmentation, the modified model stalled for the first few epochs and finished with a higher training error than CorrectionNet, so it was undertrained rather than overfit. With augmentation, it trained steadily and reached the lowest training error of all four runs (8.7 px). Each configuration was trained only once, so part of this difference could be run-to-run variation.